

MODULE II

ER MODEL

Dr. Geetha Mary A
Professor,
SCOPE, VIT, Vellore

Sources:

Pearson Education, Inc. 2011, Elmasri/Navathe, Fundamentals of Database Systems, Sixth Edition
McGraw Hill Education , 2010, Silberschatz, Korth and Sudarshan, Database System Concepts, Sixth edition

Topic Outline

- Overview of Database Design Process
- Example Database Application (COMPANY)
- ER Model Concepts
 - Entities and Attributes
 - Entity Types, Value Sets, and Key Attributes
 - Relationships and Relationship Types
 - Weak Entity Types
 - Roles and Attributes in Relationship Types
- ER Diagrams - Notation
- ER Diagram for COMPANY Schema
- Alternative Notations – UML class diagrams, others

Overview of Database Design Process

- Two main activities:
 - ▣ Database design
 - ▣ Applications design
- Focus of this topic is on database design
 - ▣ To design the conceptual schema for a database application
- Applications design focuses on the programs and interfaces that access the database
 - ▣ Generally considered part of software engineering

ER Model has detailed descriptions of

- ❑ What are the entities and relationships in the enterprise?
- ❑ What information about these entities and relationships should we store in the database?
- ❑ What are the integrity constraints or business rules that hold?

Example COMPANY Database

- We need to create a database schema design based on the following (simplified) **requirements** of the COMPANY Database:
 - The company is organized into DEPARTMENTS. Each department has a name, number and an employee who *manages* the department. We keep track of the start date of the department manager. A department may have several locations.
 - Each department *controls* a number of PROJECTs. Each project has a unique name, unique number and is located at a single location.

Example COMPANY Database (Contd.)

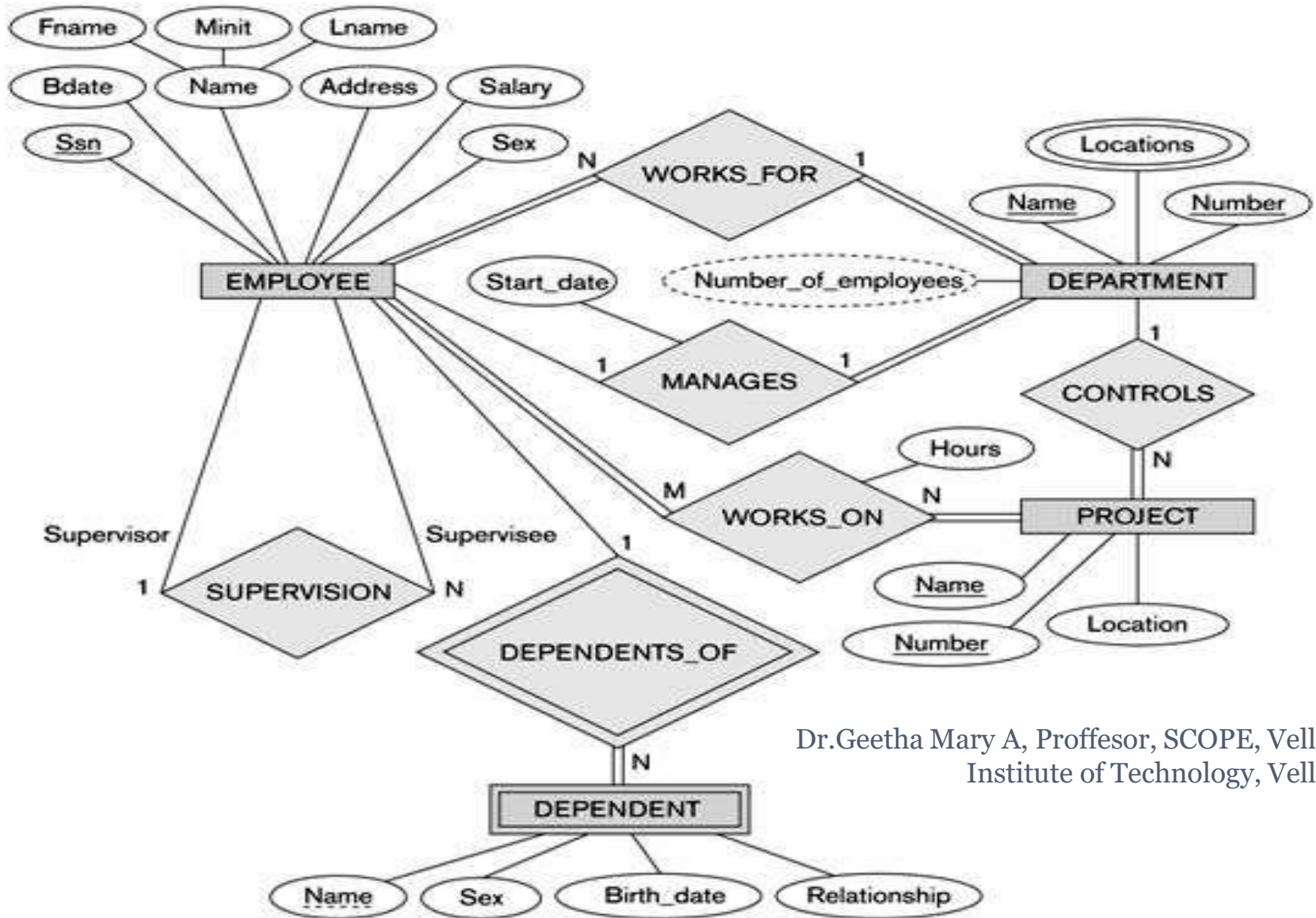
- We store each EMPLOYEE's social security number, address, salary, sex, and birthdate.
 - Each employee *works for* one department but may *work on* several projects.
 - We keep track of the number of hours per week that an employee currently works on each project.
 - We also keep track of the *direct supervisor* of each employee.
- Each employee may *have* a number of DEPENDENTS.
 - For each dependent, we keep track of their name, sex, birthdate, and relationship to the employee.

ER Model Concepts

□ Entities and Attributes

- **Entities** are specific objects or things in the mini-world that are represented in the database.
 - For example the EMPLOYEE John Smith, the Research DEPARTMENT, the ProductX PROJECT
- **Attributes** are properties used to describe an entity.
 - For example an EMPLOYEE entity may have the attributes Name, SSN, Address, Sex, BirthDate
- A specific entity will have a **value** for each of its attributes.
 - For example a specific employee entity may have Name='John Smith', SSN='123456789', Address ='731, Fondren, Houston, TX', Sex='M', BirthDate='09-JAN-55'
- Each attribute has a **value set** (or data type) associated with it – e.g. integer, string, subrange, enumerated type, ...

ER Schema diagram of company DB



Dr.Geetha Mary A, Proffesor, SCOPE, Vellore
Institute of Technology, Vellore

Entity Sets

- A *database* can be modeled as:
 - ▣ a collection of entities,
 - ▣ relationship among entities.
- An *entity* is an object that exists and is distinguishable from other objects.
 - ▣ Example: specific person, company
- Entities have *attributes*
 - ▣ Example: people have *names* and *addresses*
- An *entity set* is a set of entities of the same type that share the same properties.
 - ▣ Example: set of all persons, companies, trees, holidays

Entity Sets *customer* and *loan*

customer-id customer- customer- customer- loan- amount
name street city number

321-12-3123	Jones	Main	Harrison	L-17	1000
019-28-3746	Smith	North	Rye	L-23	2000
677-89-9011	Hayes	Main	Harrison	L-15	1500
555-55-5555	Jackson	Dupont	Woodside	L-14	1500
244-66-8800	Curry	North	Rye	L-19	500
963-96-3963	Williams	Nassau	Princeton	L-11	900
335-57-7991	Adams	Spring	Pittsfield	L-16	1300

customer *loan*

Attributes

- Entity is represented by a set of attributes(properties)

Eg:

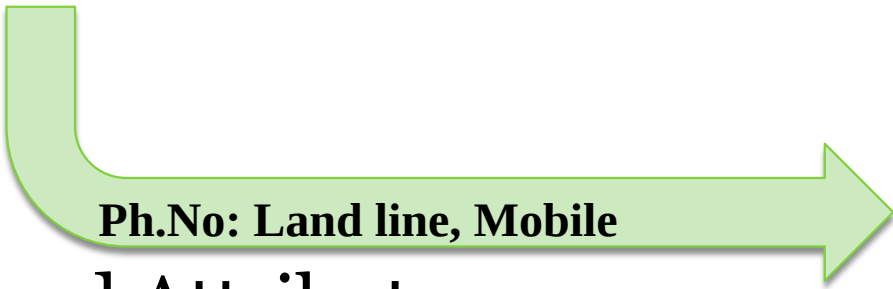
customer = (customer-id, customer-name, customer-street, customer-city)

loan = (loan-number, amount)

- *Domain* – the set of permitted values for each attribute

Attribute Types

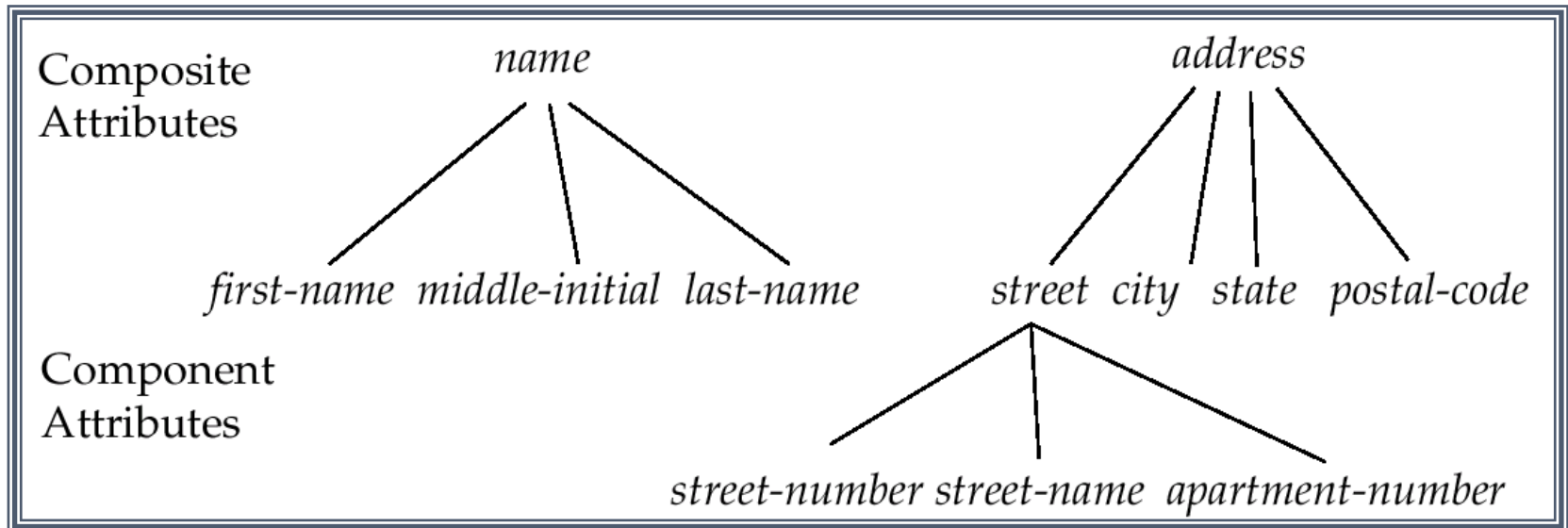
- Simple and Composite
- Single valued and multi valued



Ph.No: Land line, Mobile

- Stored versus Derived Attributes:
 - Age → calculated from DOB
 - DOB → Stored attribute

Composite Attributes



Grouping together related attributes

Attribute Types

- **Null Values:**

- College Degrees of person, apartment number

- **Complex Attributes:**

- Composite attribute and multivalued attributes are nested in an arbitrary way.
- Composite attribute → parentheses ()
- Multivalued attribute → braces { }
- Eg:

```
{AddressPhone( { Phone(AreaCode,PhoneNumber)},  
Address(streetAddress(number,street,apartmentNumber),  
city,state,zip) ) }
```

Entity Types and Key Attributes

- Entities with the same basic attributes are grouped or typed into an entity type.
 - For example, the entity type EMPLOYEE and PROJECT.
- An attribute of an entity type for which each entity must have a unique value is called a key attribute of the entity type.
 - For example, SSN of EMPLOYEE.

Entity Types and Key Attributes

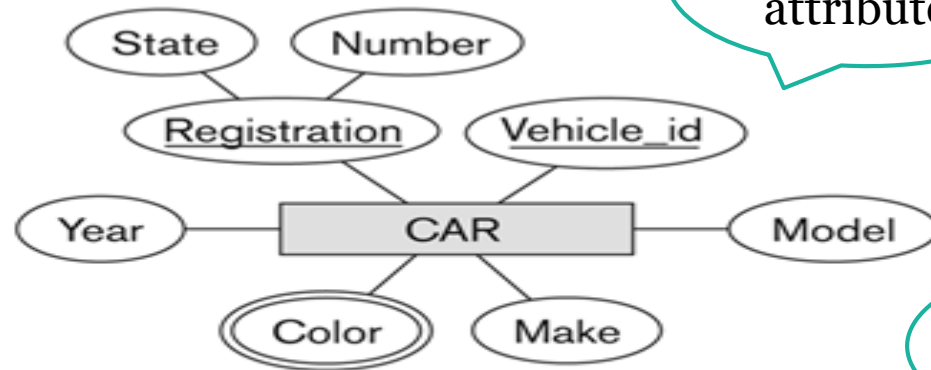
- A key attribute may be composite.
 - VehicleTagNumber is a key of the CAR entity type with components (Number, State).
- An entity type may have more than one key.
 - The CAR entity type may have two keys:
 - VehicleIdentificationNumber (popularly called VIN)
 - VehicleTagNumber (Number, State),
- Each key is underlined

Displaying an Entity type

- In ER diagrams, an entity type is displayed in a rectangular box
- Attributes are displayed in ovals
 - Each attribute is connected to its entity type
 - Components of a composite attribute are connected to the oval representing the composite attribute
 - Each key attribute is underlined
 - Multivalued attributes displayed in double ovals
- See CAR example on next slide

Entity Type CAR with two keys and a corresponding Entity Set

(a)



(b)

CAR
Registration (Number, State), Vehicle_id, Make, Model, Year, {Color}

CAR₁
((ABC 123, TEXAS), TK629, Ford Mustang, convertible, 2004 {red, black})

CAR₂
((ABC 123, NEW YORK), WP9872, Nissan Maxima, 4-door, 2005, {blue})

CAR₃
((VSY 720, TEXAS), TD729, Chrysler LeBaron, 4-door, 2002, {white, blue})

⋮

Entity Set

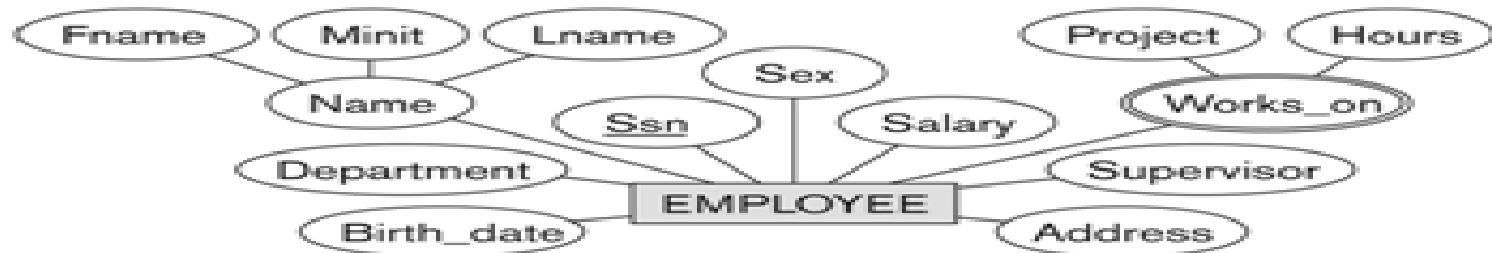
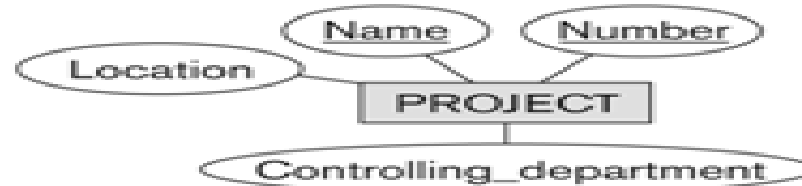
- Each entity type will have a collection of entities stored in the database
 - ▣ Called the **entity set**
- Previous slide shows three CAR entity instances in the entity set for CAR
- Same name (CAR) used to refer to both the entity type and the entity set
- Entity set is the current *state* of the entities of that type that are stored in the database

Initial Design of Entity Types for the COMPANY Database Schema

- Based on the requirements, we can identify four initial entity types in the COMPANY database:
 - ▣ DEPARTMENT
 - ▣ PROJECT
 - ▣ EMPLOYEE
 - ▣ DEPENDENT
- Their initial design is shown on the following slide
- The initial attributes shown are derived from the requirements description

Initial Design of Entity Types:

EMPLOYEE, DEPARTMENT, PROJECT, DEPENDENT



Weak Entity Types

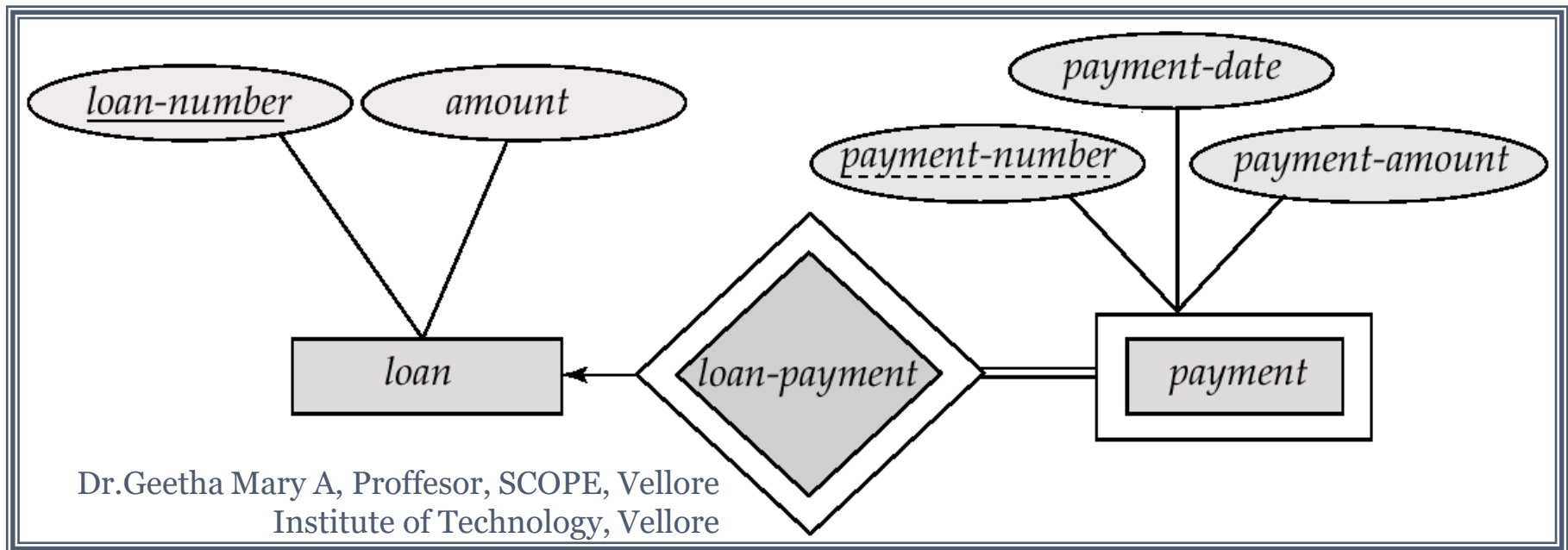
- An entity that does not have a key attribute
- A weak entity must participate in an identifying relationship type with an owner or identifying entity type
- Entities are identified by the combination of:
 - A partial key of the weak entity type
 - The particular entity they are related to in the identifying entity type
- **Example:**
 - A DEPENDENT entity is identified by the dependent's first name, *and* the specific EMPLOYEE with whom the dependent is related
 - Name of DEPENDENT is the *partial key*
 - DEPENDENT is a *weak entity type*
 - EMPLOYEE is its identifying entity type via the identifying relationship type DEPENDENT_OF

Weak Entity Sets

- Is an entity set that does not have a primary key
 - Payment-number, payment date, payment amount
- Must be associated with a *identifying entity set*
 - *Loan*
 - Identifying relationship depicted using a double diamond
- The *discriminator* is a set of attributes that allows the distinction.
 - Payment-number
- The primary key of a weak entity set is formed by the primary key of the identifying entity set , plus the weak entity set's discriminator.
 - {loan-number,payment-number}

Weak Entity Sets (Cont.)

- weak entity set is depicted by double rectangles.
- discriminator of a weak entity set is underlined with a dashed line.
- *payment-number* – discriminator of the *payment* entity set
- Primary key for *payment* – (*loan-number*, *payment-number*)



Entity-Relationship (E-R) Modeling

□ Relationship

- An association between the instances of one or more entity types that is of interest to the organization
- Association indicates that an event has occurred or that there is a natural link between entity types
- Relationships are always labeled with verb phrases

Naming and Defining Relationships

- Relationship name is a verb phrase
- Avoid vague names
- Guidelines for defining relationships
 - Definition explains what action is being taken and why it is important
 - Give examples to clarify the action
 - Optional participation should be explained
 - Explain reasons for any explicit maximum cardinality

Naming and Defining Relationships

- Guidelines for defining relationships
 - ▣ Explain any restrictions on participation in the relationship
 - ▣ Explain extent of the history that is kept in the relationship
 - ▣ Explain whether an entity instance involved in a relationship instance can transfer participation to another relationship instance

Relationships



- Relationship Types and Sets
- Relationship Degree
- Entity Roles and Recursive Relationships
- Relationship Constraints
- Attributes of Relationship Types

Relationship Types and Sets

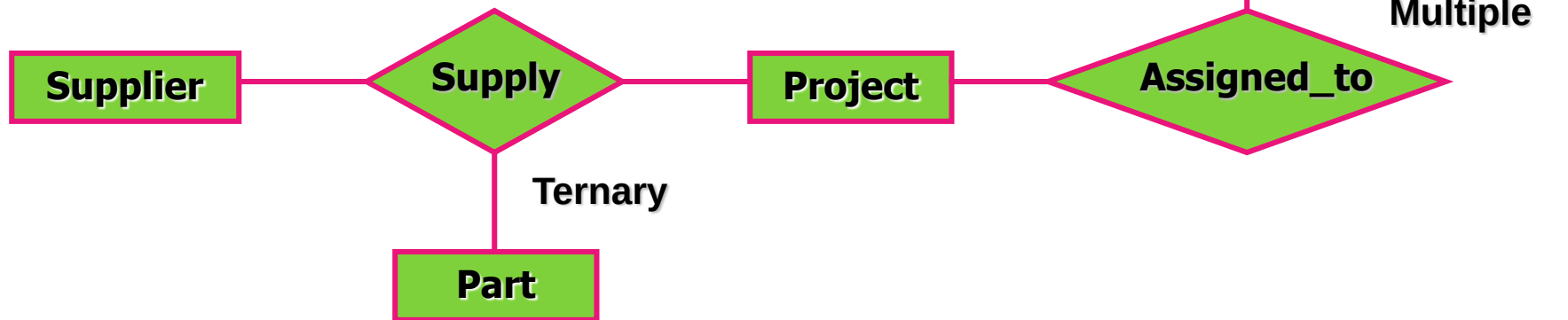
A Relationship is an association among two or more entities
(e.g John works in Pharmacy department)

- A Relationship Type defines the relationship, and a Relationship Set represents a set of relationship instances
- A Relationship Type thus defines the structure of the Relationship Set

Relationship Type and corresponding Set are customarily referred to by the **same** name

Relationship Degree

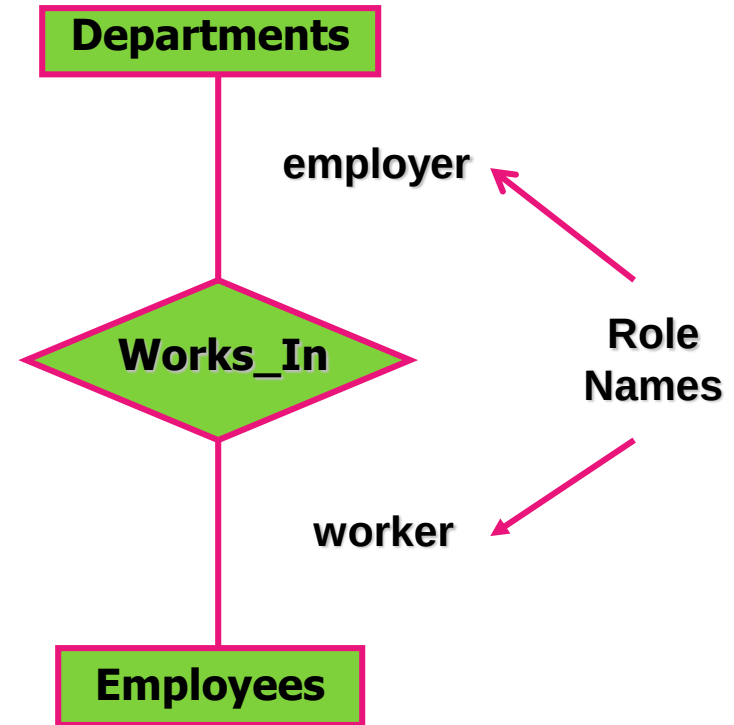
- The degree of a relationship type is the number of participating entity types
 - 2 entities: *Binary Relationship*
 - 3 entities: *Ternary Relationship*
 - n entities: *N-ary Relationship*
 - Same entity type could participate in multiple relationship types



Entity Roles

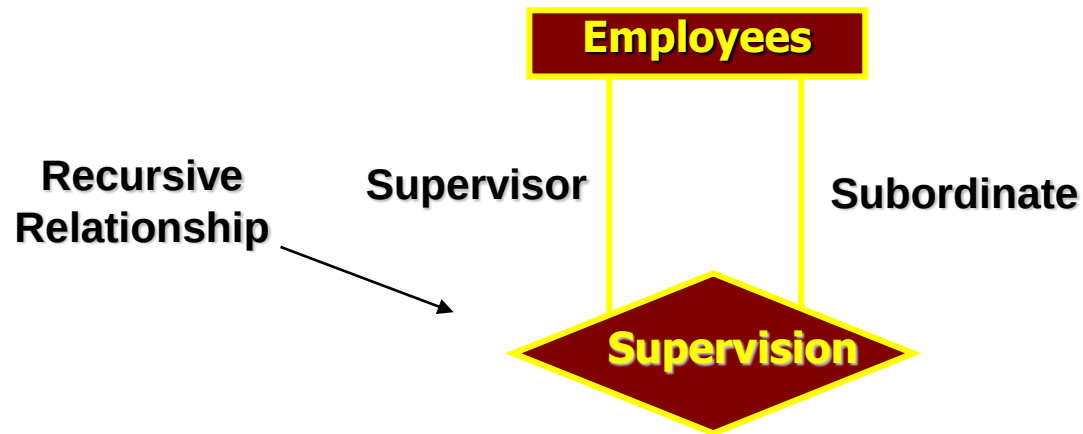
- Each entity type that participates in a relationship type plays a particular role in the relationship type
- The role name signifies the role that a participating entity from the entity type plays in each relationship instance, i.e. it explains what the relationship

means



Recursive Relationships

- Same entity type can participate more than once in the same relationship type under different “roles”
- Such relationships are called “**Recursive Relationships**”



Relationship Constraints



What are Relationship Constraints ?

- Constraints on the relationship type limit the possible combination of entities that may participate in the corresponding relationship set

Kinds of Constraints

What kind of constraints can be defined in the ER Model?

□ Cardinality Constraints

□ Participation Constraints

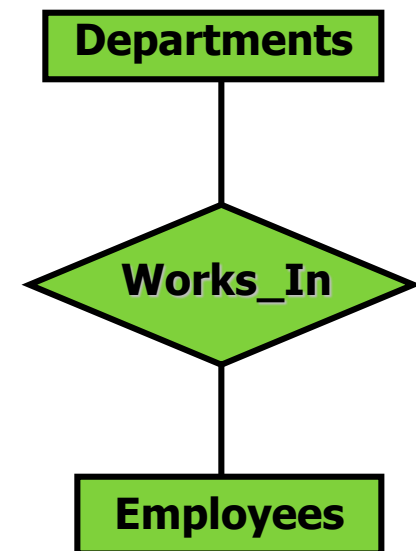
Together called “Structural Constraints”

Constraints are represented by
specific notation in the ER diagram

Possible Cardinality Ratios

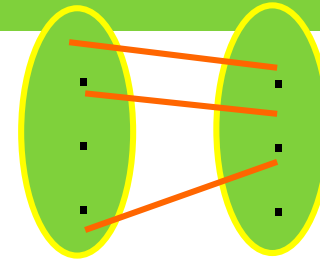
- The “Cardinality Ratio” for a binary relationship specifies the number of relationship instances that an entity can participate in

- Works-In is a binary relationship
- Participating entities are
DEPARTMENT : EMPLOYEE
- *One* department can have
Many employees -
Cardinality Ratio is 1 : N

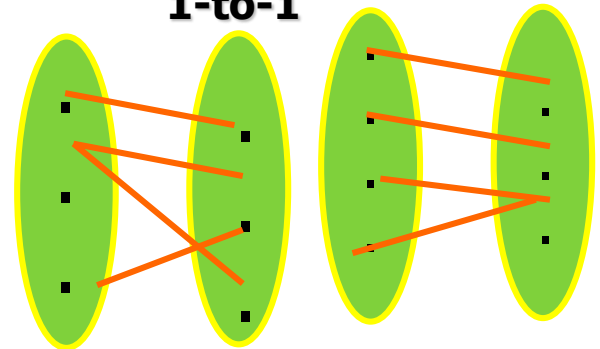


Possible Cardinality Ratios

- 1-to-1 (1 : 1)
 - Both entities can participate in only **one** relationship instance
- 1-to-Many, Many-to-1 (1 : N, N : 1)
 - One entity can participate in **many** relationship instances
- Many-to-Many (N: M)
 - Both entities can participate in **many** relationship instance

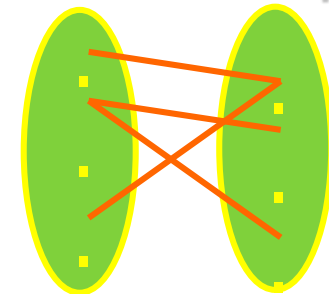


1-to-1



1- to - Many

Many - to - 1



Many-to-Many

Example Cardinality Constraints

How many Employees can work in a Department?

One employee can work in only one department

How many Employees can be employed by a Department?

One department can employ many employees

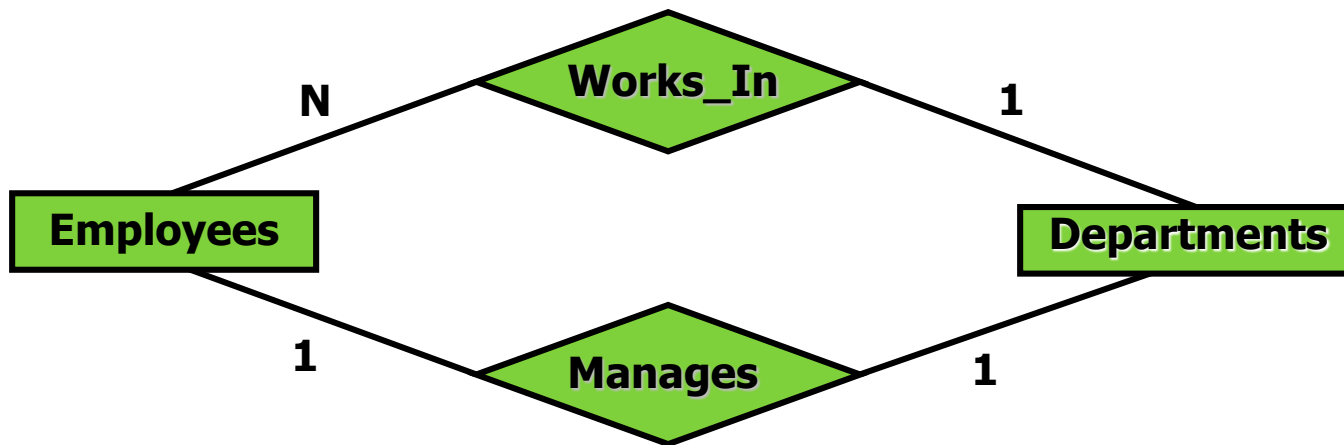
How many managers can a department have?

One department can have only one manager

How many departments can an employee manage?

One employee can have manage only one department

Representing Cardinality



One employee can work in only one department

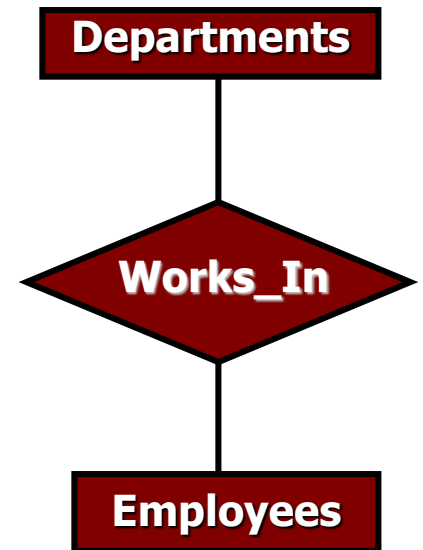
One department can employ many employees

One department can have only one manager

One employee can manage only one department

Existence Dependency

- Existence dependency indicates whether the existence of an entity depends on its relationship to another entity via the relationship type
 - Every employee must work for a department - EMPLOYEE is existentially dependent on DEPARTMENT via the Works In relationship type



Kinds of participating constraints



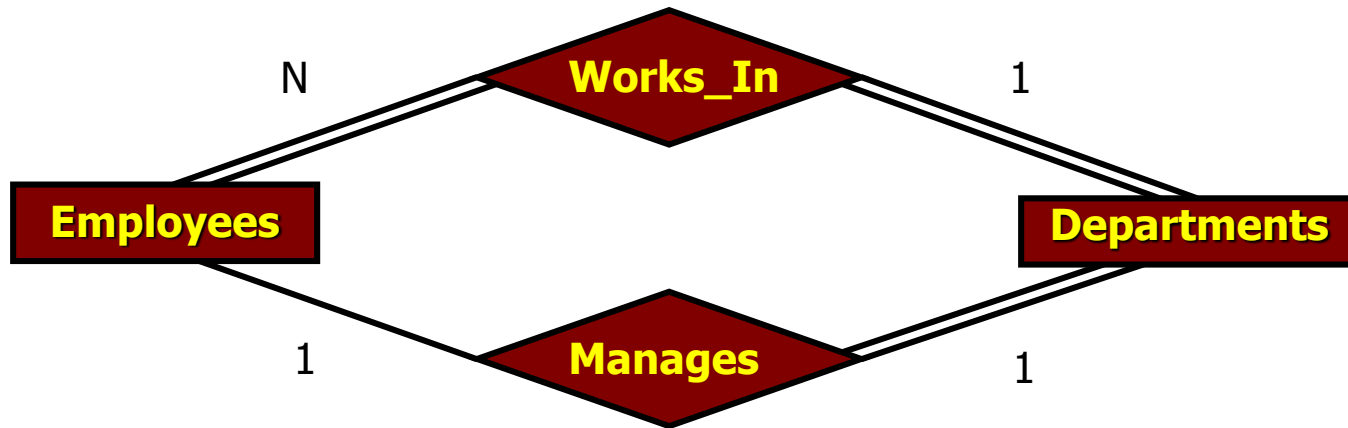
□ TOTAL Participation (Existence Dependency)

Constraint : Every employee must work for a department

□ PARTIAL Participation

Constraint : Not every employee is a manager

Representing Participation



Every employee must work for a department

Every department must have a manager

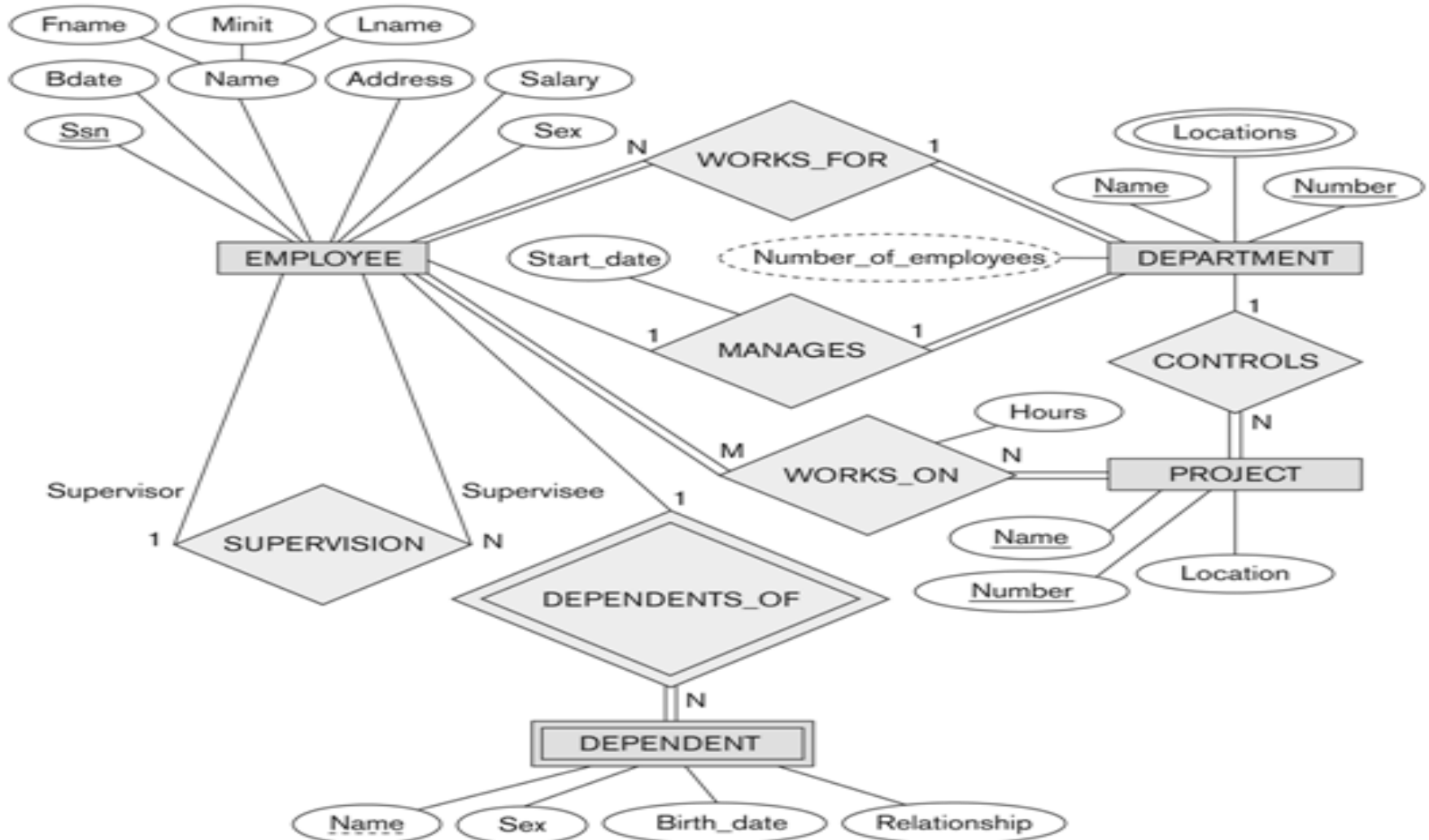
Every department must have employees

Not every employee is a manager

Attributes of Relationship types

- A relationship type can have attributes:
 - ▣ For example, HoursPerWeek of WORKS_ON
 - ▣ Its value for each relationship instance describes the number of hours per week that an EMPLOYEE works on a PROJECT.
 - A value of HoursPerWeek depends on a particular (employee, project) combination
 - ▣ Most relationship attributes are used with M:N relationships
 - In 1:N relationships, they can be transferred to the entity type on the N-side of the relationship

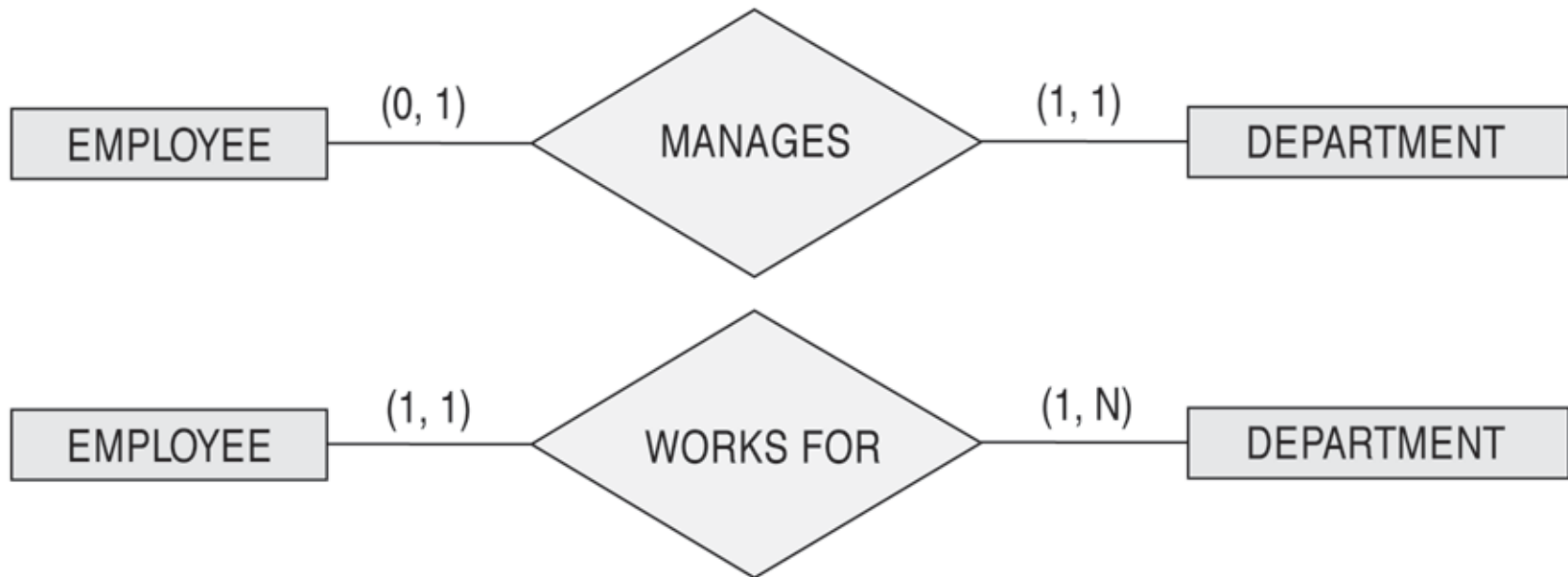
Recursive Relationship Type is: SUPERVISION (participation role names are shown)



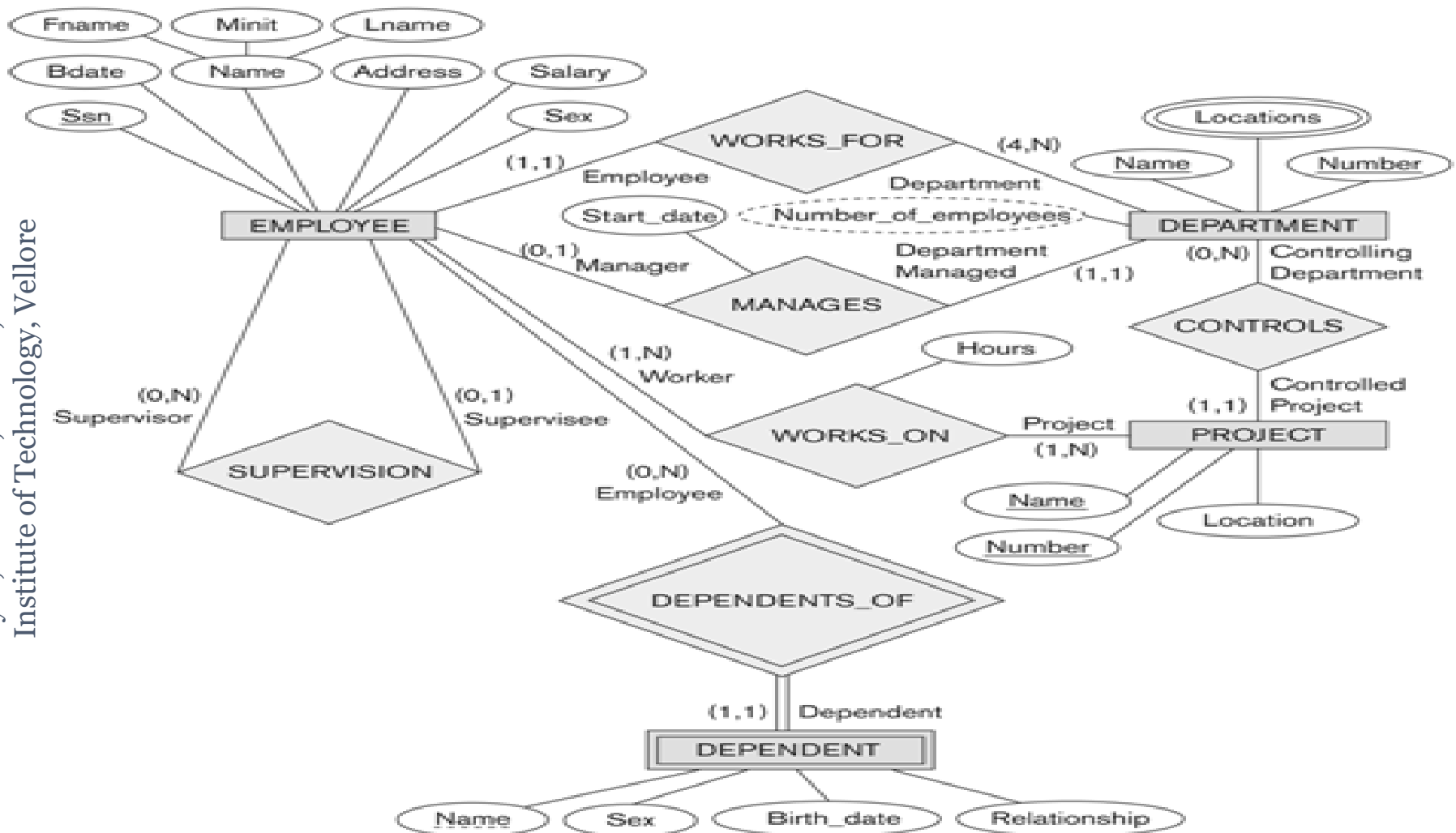
Alternative (min, max) notation for relationship structural constraints:

- Specified on each participation of an entity type E in a relationship type R
- Specifies that each entity e in E participates in at least *min* and at most *max* relationship instances in R
- Default(no constraint): min=0, max=n (signifying no limit)
- Must have $\text{min} \leq \text{max}$, $\text{min} \geq 0$, $\text{max} \geq 1$
- Derived from the knowledge of mini-world constraints
- Examples:
 - A department has exactly one manager and an employee can manage at most one department.
 - Specify (0,1) for participation of EMPLOYEE in MANAGES
 - Specify (1,1) for participation of DEPARTMENT in MANAGES
 - An employee can work for exactly one department but a department can have any number of employees.
 - Specify (1,1) for participation of EMPLOYEE in WORKS_FOR
 - Specify (0,n) for participation of DEPARTMENT in WORKS_FOR

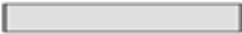
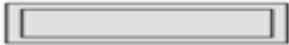
The (min,max) notation for relationship constraints



COMPANY ER Schema Diagram using (min, max) notation



Summary of notation for ER diagrams

Symbol	Meaning
	Entity
	Weak Entity
	Relationship
	Identifying Relationship
	Attribute
	Key Attribute
	Multivalued Attribute
	Composite Attribute
	Derived Attribute
	Total Participation of E_2 in R
	Cardinality Ratio 1 : N for $E_1:E_2$ in R
	Structural Constraint (min, max) on Participation of E in R

Alternative diagrammatic notation



- ER diagrams is one popular example for displaying database schemas
- Many other notations exist in the literature and in various database design and modeling tools
- UML class diagrams is representative of another way of displaying ER concepts that is used in several commercial design tools